

A COMPLETE PRACTITIONER'S GUIDE

# Advanced Retrieval- Augmented Generation

---

From First Principles to Production-Grade  
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE SIX

## Advanced Retrieval Techniques

Beginner to Industry Level • Assumes only basic Python

## MODULE SIX

# Advanced Retrieval Techniques

---

*The retriever from Module 5 takes the user's question exactly as typed and returns whole chunks exactly as stored. Both of those defaults leave quality on the table. This module makes the retriever genuinely intelligent. You will learn to transform the query before searching — with multi-query retrieval, query rewriting and expansion, and the self-query retriever that turns natural language into structured metadata filters — and to transform the documents after searching — with the parent document retriever, contextual compression, and two-stage re-ranking. Together these techniques are what separate a demo-grade RAG system from an enterprise-grade one.*

### WHERE WE ARE

Module 5 gave you a solid retriever: hybrid search, score thresholds, and MMR. But it had two blind spots — it trusted the user's exact wording, and it returned chunks unmodified. This module fixes both. Every technique here still produces the same standard retriever interface ( `retriever.invoke(query)` ), so it slots into your existing chain unchanged. Module 7 then builds full RAG *patterns* on top of these retrievers.

## 6.1 Introduction

---

By the end of Module 5 you had a competent retriever. It searched by meaning and by exact words, it could decline to answer when nothing fit, and it avoided returning four copies of the same fact. For many projects that is genuinely enough.

But that retriever still makes two silent assumptions, and both cost you quality.

**Assumption one: the user's query is a good search query.** It often is not. Real users write queries that are vague ("*tell me about the leave thing*"), under-specified, full of pronouns, or phrased completely differently from how the answer is written in the documents. The retriever embeds whatever the user typed and searches with it. If the query is a poor representation of the user's actual information need, retrieval starts from a weak foundation no matter how good the search algorithm is. **Garbage query in, garbage chunks out.**

**Assumption two: a retrieved chunk should be used exactly as stored.** Also often false. A chunk that the retriever correctly identified as relevant may still be mostly irrelevant *text* — five useful sentences buried in a thousand-character chunk of surrounding material. Feeding the whole chunk to the model wastes context-window space and surrounds the real answer with distracting noise. And the chunk size that retrieves *precisely* is often too small to give the model enough *context* to answer well.

Advanced retrieval is the discipline of removing these two assumptions. Every technique in this module intervenes at one of two points: it either **improves the query before the search happens**, or it **improves the documents after the search happens**. That single distinction — pre-retrieval versus post-retrieval — is the organising idea of the entire module, and it is worth fixing firmly in mind before we go further.

This module covers, on the pre-retrieval side: multi-query retrieval, query rewriting, query expansion, and the self-query retriever. On the post-retrieval side: the parent document retriever, contextual compression, and re-ranking. Together they turn a competent retriever into an enterprise-grade one.

## 6.2 Why This Topic Matters

### WHY THIS MATTERS

In Module 5 you learned that retrieval quality caps system quality. This module is where you push that cap upward. The techniques here routinely turn a RAG system that "mostly works" into one that works reliably on messy, real-world queries over large, structured corpora. They are also exactly what enterprise RAG systems are built from — and exactly what senior interviewers probe with questions like "how would you handle a vague user query?" or "what is the parent document retriever and what problem does it solve?"

Four concrete reasons this module repays close study:

1. **Real users do not write clean queries.** A retriever that only works on well-formed questions fails in production. Query transformation is how you cope with the queries you actually get.
2. **It resolves the chunk-size dilemma.** Module 2 forced a painful trade-off — small chunks retrieve precisely but lack context; large chunks have context but retrieve imprecisely. The parent document retriever dissolves that trade-off entirely.
3. **It cuts cost and noise.** Contextual compression and re-ranking strip retrieved results down to what genuinely matters, lowering token cost and sharpening the model's focus.
4. **It is the bridge to structured data.** The self-query retriever lets natural-language questions exploit structured metadata — the meeting point of semantic search and database querying, and the foundation of metadata-aware enterprise retrieval.

## 6.3 A Beginner-Friendly Explanation

---

Picture the Module 5 retriever as a fast but very literal library clerk. You hand the clerk a slip of paper with your question on it; the clerk searches for exactly what is written and brings back whole books. Fast, but literal.

Advanced retrieval replaces the literal clerk with a thoughtful **research assistant** who does five smarter things.

**Before searching, the assistant improves your question.** If your slip says "*tell me about the leave thing*," the assistant does not search for that vague phrase. Instead it thinks: "*the user probably means one of: annual leave policy, sick leave, parental leave*" and searches for **all of those**, then pools the results. This is **multi-query retrieval** — turning one weak query into several strong ones.

**The assistant also rephrases clumsy questions into clean ones.** If your question is awkwardly worded, the assistant first rewrites it into the clear, keyword-rich form the documents are likely to use, *then* searches. This is **query rewriting and expansion**.

**The assistant notices when your question hides a filter.** If you ask "*what HR policies were updated after 2023?*", the assistant recognises two parts: a topic to search for ("HR policies") and a *condition* to filter on ("updated after 2023"). It searches for the topic but restricts the search to documents matching the condition. This is the **self-query retriever** — it reads your sentence and automatically writes a metadata filter.

**After searching, the assistant gives you the right amount of context.** It pinpoints the exact relevant *paragraph* — precise — but then hands you the whole *chapter* that paragraph sits in, so you have enough surrounding context to understand it. This is the **parent document retriever** — "find small, return big."

**Finally, the assistant trims and re-sorts what it found.** It reads the books it pulled, highlights only the sentences that actually answer your question (discarding the rest), and re-reads its shortlist carefully to put the single best result on top. This is **contextual compression** and **re-ranking**.

That is the whole module: a retriever that thinks before it searches and curates after it searches.

### DEFINITION

**Pre-retrieval techniques** transform or expand the user's *query* before the search runs, so the search starts from a better representation of the user's need. **Post-retrieval techniques** transform, trim, or re-order the retrieved *documents* after the search runs, so the model receives a cleaner, better-ordered set of context.

## 6.4 Real-World Analogy: The Literal Clerk vs the Research Assistant

### REAL-WORLD ANALOGY

You walk into a law library and say to the clerk: *"I need the stuff about the contract problem from a few years back."*

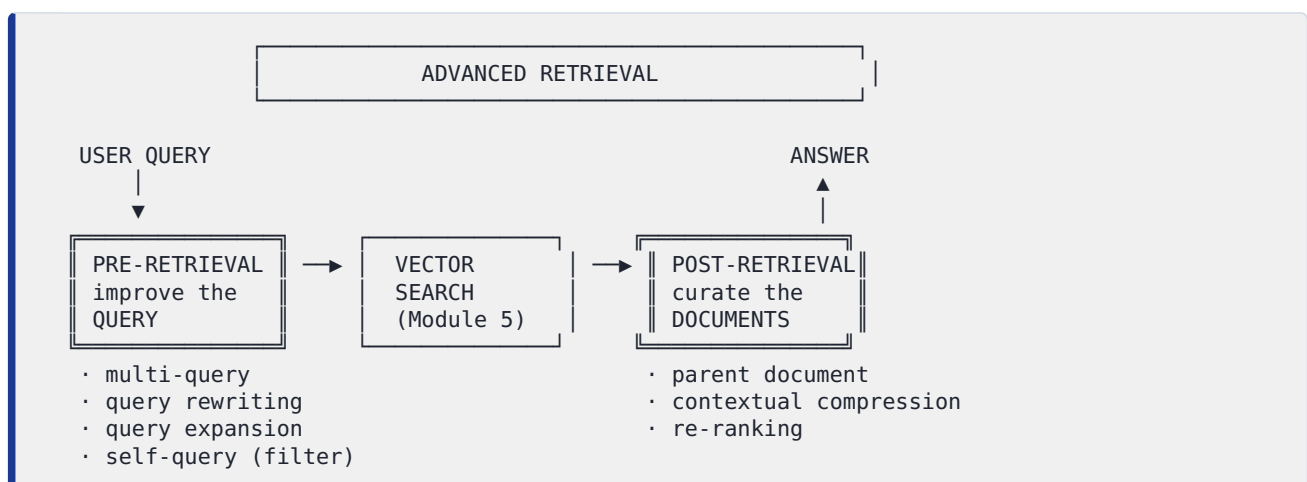
The **literal clerk** (the Module 5 retriever) walks to the catalogue, searches for the exact words "stuff," "contract problem," and "few years back," and brings back whatever loosely matches — whole volumes, some relevant, some not.

The **research assistant** (the Module 6 retriever) does five things differently. They *ask the question better*: "you likely mean breach-of-contract case law." They *generate variations*: they look up "breach of contract," "contract dispute," and "contractual non-performance" and pool the results. They *spot the hidden filter*: "a few years back" means they restrict the search to cases from the relevant date range. They *give the right context*: they find the exact paragraph of the exact ruling, but photocopy the whole section around it so it makes sense. And they *curate*: they highlight only the passages that bear on your problem and lay the most on-point ruling on top of the pile.

Same library, same catalogue — a transformed result, because the assistant intervened both *before* the search (improving the question) and *after* it (curating the findings).

## 6.5 Core Concepts: Two Points of Intervention

Every advanced retrieval technique fits one of two categories, defined by *when* it acts relative to the vector search.



- **Pre-retrieval (query-side) techniques** accept that the user's raw query is often a poor search input and *fix it first*. They rewrite it, generate variations of it, or extract a structured filter from it. The search itself is unchanged — it simply receives a better query.

- **Post-retrieval (document-side) techniques** accept that raw retrieved chunks are often imperfect context and *curate them after the fact*. They swap small chunks for their larger parents, strip out irrelevant text, or re-order the results by a more accurate measure of relevance.

A crucial practical point: **these techniques compose**. A production retriever might run multi-query *and* a self-query filter on the way in, and contextual compression *and* re-ranking on the way out. They are not alternatives; they are layers. We will study each in isolation, then combine them in Section 6.12.

## 6.6 Multi-Query Retrieval

---

A single query is a single point in embedding space. If the user phrased their need slightly differently from how the answer is written, that single point lands in slightly the wrong place, and the right chunk sits just outside the top- $k$ . There is no single "correct" phrasing of an information need — and betting everything on the user's one phrasing is fragile.

**Multi-query retrieval** removes that fragility. It uses an LLM to generate **several alternative phrasings** of the user's question, runs retrieval for *each* of them, and then pools and de-duplicates the combined results. Where the user's wording lands in the wrong place, one of the LLM's rephrasings lands in the right place — and the right chunk is recovered.

For the query "How do I get time off work?", the LLM might generate: "What is the annual leave policy?", "How do I request vacation days?", and "What is the procedure for taking paid leave?" Each is searched; the union of their results, with duplicates removed, becomes the final chunk set. The retriever has effectively cast a wider, smarter net.

```
from langchain.retrievers.multi_query import MultiQueryRetriever

multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,    # any Module 5 retriever
    llm=llm,                    # the LLM that generates query variations
)
```

The cost is real and worth naming: multi-query runs  $N$  searches instead of one and makes an extra LLM call to generate the variations, so it adds latency and a little expense. It is most valuable when queries are short, vague, or vocabulary-mismatched with the corpus.

**WHY THIS MATTERS**

Multi-query retrieval directly attacks *recall* — the chance that the genuinely relevant chunk is found at all. A single phrasing is a single roll of the dice; multiple phrasings are several rolls, and you keep the best of all of them. This is why multi-query is one of the most reliable upgrades for systems where users ask the same thing in many different ways — which is to say, almost all real systems.

## 6.7 Query Rewriting and Query Expansion

---

Multi-query generates *whole alternative questions*. Two related, lighter techniques transform the *single* query in place.

**Query rewriting** uses an LLM to turn a messy query into a clean one before searching. Its jobs include: resolving pronouns and references (in a conversation, "*how much does it cost?*" is rewritten to "*how much does the premium plan cost?*" using earlier context), removing conversational filler ("*hey so I was wondering, what's the deal with refunds*" becomes "*what is the refund policy*"), and re-phrasing the query into the vocabulary and style the documents actually use. A query that matches the documents' own language retrieves far better.

**Query expansion** *adds* useful terms to the query rather than replacing it — synonyms, related concepts, likely keywords. Expanding "*car insurance*" with "*vehicle, automobile, motor policy, coverage*" widens the query's footprint so it overlaps more chunks. (Multi-query retrieval can be seen as a structured, productised form of query expansion — generating whole expanded queries instead of loose extra terms.)

Both are simple to implement: an LLM call with a focused prompt, run *before* retrieval.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

rewrite_prompt = ChatPromptTemplate.from_template(
    "Rewrite the user's question into a single, clear, keyword-rich "
    "search query. Resolve any vague references. Return ONLY the "
    "rewritten query.\n\nUser question: {question}"
)
query_rewriter = rewrite_prompt | llm | StrOutputParser()

clean_query = query_rewriter.invoke({"question": "hey whats the deal w refunds"})
chunks = base_retriever.invoke(clean_query)  # search with the clean query
```

**BEST PRACTICE**

Query rewriting is essential the moment your RAG system becomes *conversational*. In a multi-turn chat, a follow-up like "what about the annual one?" is meaningless to a retriever on its own — it has no idea what "the annual one" refers to. A rewriting step that folds in the conversation history (turning it into "what is the annual leave policy?") is what makes follow-up questions work at all. Treat query rewriting as mandatory for chat-style RAG, not optional.

## 6.8 The Self-Query Retriever: Metadata-Aware Retrieval

This is the technique that bridges semantic search and structured database querying, and it is the most conceptually important in the module.

Recall from Module 2 that every chunk carries **metadata** — fields like `year`, `author`, `department`, `doc_type`, `rating`. Module 2 showed you could *filter* on metadata, but only if you wrote the filter in code. The **self-query retriever** removes that limitation: it uses an LLM to **read the user's natural-language question and automatically write the metadata filter itself**.

Consider the question: "What HR policy documents were published after 2023 about remote work?" This sentence secretly contains two different kinds of search:

- A **semantic** part — find chunks *about* "remote work."
- A **structured filter** part — restrict to chunks where `department = "HR"`, `doc_type = "policy"`, and `year > 2023`.

A plain retriever embeds the whole sentence and searches semantically — and the phrase "published after 2023" just becomes fuzzy meaning, not an actual constraint. The self-query retriever instead uses the LLM to **decompose** the question into its two parts: it extracts a clean semantic query ("remote work") **and** a structured filter (`year > 2023 AND department = "HR" AND doc_type = "policy"`). It then runs a *filtered* similarity search — semantic search over only the chunks that satisfy the filter.

To do this, the self-query retriever needs to be told, in advance, **what metadata fields exist and what they mean** — a schema. You provide a short description of each field; the LLM uses those descriptions to know what it is allowed to filter on.



```

from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain.chains.query_constructor.base import AttributeInfo

# Describe the metadata schema so the LLM knows what it can filter on
metadata_field_info = [
    AttributeInfo(name="department", description="Owning department: "
                  "HR, Finance, or Engineering", type="string"),
    AttributeInfo(name="doc_type", description="Type of document: "
                  "policy, report, or memo", type="string"),
    AttributeInfo(name="year", description="Year the document was "
                  "published", type="integer"),
]
document_content_description = "Internal company documents and policies"

self_query_retriever = SelfQueryRetriever.from_llm(
    llm=llm,
    vectorstore=vector_store,
    document_contents=document_content_description,
    metadata_field_info=metadata_field_info,
)
# The LLM now extracts both a semantic query AND a filter from the question
chunks = self_query_retriever.invoke(
    "HR policies about remote work published after 2023"
)

```

### INTERVIEW INSIGHT

"What is the self-query retriever and what problem does it solve?" is a favourite advanced question. The complete answer: it uses an LLM to decompose a natural-language question into two parts — a *semantic query* (for vector similarity search) and a *structured metadata filter* — then runs a filtered similarity search. It solves the problem that constraints like dates, categories, authors, or numeric thresholds are *filtering* conditions, not *semantic* ones; a plain retriever treats "after 2023" as vague meaning, whereas the self-query retriever turns it into a real `year > 2023` filter. It requires a declared metadata schema so the LLM knows which fields it may filter on. It is, in effect, the bridge between unstructured semantic search and structured database querying.

### COMMON MISTAKE

Expecting the self-query retriever to filter on metadata you never attached during ingestion. The LLM can only build filters over fields that (a) actually exist on your chunks and (b) you described in `metadata_field_info`. If your chunks have no `year` field, no amount of clever questioning produces a date filter. This is the practical pay-off of Module 2's insistence on designing a metadata schema *before* ingestion — metadata-aware retrieval is impossible without metadata-aware ingestion.

## 6.9 The Parent Document Retriever

This technique resolves the chunk-size dilemma from Module 2 — and once you see it, the resolution feels obvious.

**The dilemma, restated.** Small chunks make *precise* retrieval: a small chunk has a sharp, single-meaning vector, so similarity search pinpoints it accurately (Module 3's "one

chunk, one vector" principle). But a small chunk often lacks the surrounding *context* the LLM needs to write a good answer. Large chunks are the reverse: rich in context for the model, but their vectors are blurry averages that retrieve imprecisely. You seem forced to choose precision *or* context.

**The resolution: decouple the unit you search from the unit you return.** The **parent document retriever** splits each document *twice*:

- Into large **parent chunks** (for context).
- Into small **child chunks** (for precise search). Each child knows which parent it came from.

Only the *small child chunks* are embedded and stored in the vector store — so retrieval is precise. But when a child chunk is matched, the retriever does **not** return the child. It looks up and returns the child's **parent** — the larger chunk — so the LLM receives full context.

**Search small, return big.** Precision *and* context, with the trade-off dissolved rather than balanced.

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_text_splitters import RecursiveCharacterTextSplitter

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000) # context
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400) # precision

parent_retriever = ParentDocumentRetriever(
    vectorstore=vector_store, # stores the small child chunks (embedded)
    docstore=InMemoryStore(), # stores the large parent chunks
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
parent_retriever.add_documents(documents) # splits both ways automatically
chunks = parent_retriever.invoke("How is annual leave accrued?")
# → matched a small child chunk, but returns its large parent for context
```

Notice the architecture: there are **two** stores. The vector store holds the small embedded children for searching; a separate **document store** (here `InMemoryStore`; in production a persistent key-value store) holds the large parents for returning. The retriever links them by a parent ID on each child.

## 6.10 Contextual Compression

Even a correctly retrieved chunk is rarely *all* signal. A relevant 1000-character chunk might contain three sentences that answer the question and seven that do not. Feeding the whole chunk to the model means the answer arrives wrapped in noise — and that noise costs tokens and can distract the model.

**Contextual compression** is a post-retrieval step that, *given the query*, trims or filters the retrieved chunks so only the genuinely relevant content survives. The word "contextual" is important: the compression is done *in the context of the specific query* — it is not generic summarisation, it is "keep only what this question needs." There are three common compressor types:

- **Extractive compression** ( `LLMChainExtractor` ) — an LLM reads each retrieved chunk and extracts *only the sentences relevant to the query*, discarding the rest. The chunk shrinks to its useful core.
- **Filtering compression** ( `LLMChainFilter` ) — an LLM judges each chunk as a whole: relevant or not. Irrelevant chunks are dropped entirely. Cheaper than extraction; coarser.
- **Embedding-based filtering** ( `EmbeddingsFilter` ) — drops chunks whose embedding similarity to the query falls below a threshold. No LLM call, so it is fast and cheap, though less precise than an LLM judge.

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)  # extract relevant sentences

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever,  # any Module 5/6 retriever
)
chunks = compression_retriever.invoke("How many sick days do I get?")
# → each returned chunk is trimmed to only the query-relevant sentences
```

The benefit is a cleaner, denser context: less noise for the model, fewer tokens to pay for. The cost is that LLM-based compressors add an extra LLM call per chunk, which adds latency and expense — so for latency-sensitive systems, the cheap `EmbeddingsFilter` is often the pragmatic choice.

## 6.11 Re-ranking: Two-Stage Retrieval

Re-ranking is the most powerful post-retrieval technique, and understanding *why* it works requires one new distinction.

The embedding-based retrieval you have used so far is a **bi-encoder**: the query and each chunk are embedded *separately, in advance*, and compared by a simple distance metric. This is fast — chunk vectors are precomputed and stored — and it scales to millions of chunks. But because the query and chunk are never *looked at together*, the relevance estimate is somewhat coarse.

A **cross-encoder** is a different kind of model: it takes the query and one chunk *together* as a single input and outputs a single, highly accurate relevance score. Because it

examines the pair jointly, it judges relevance far more precisely than a bi-encoder. But it is slow — it cannot precompute anything, and it must run once *per chunk* — so it cannot be used to search a whole corpus.

The solution combines their strengths in **two-stage retrieval, also called retrieve-then-rerank**:

1. **Stage 1 — retrieve (fast, broad).** Use the fast bi-encoder retriever to pull a *large* candidate set — say the top 25 chunks. This stage prioritises recall: cast a wide net cheaply.
2. **Stage 2 — re-rank (slow, precise).** Run the slow, accurate cross-encoder on just those 25 candidates, scoring each, and keep only the top 4. This stage prioritises precision: carefully sort a small shortlist.

You get the bi-encoder's scalability *and* the cross-encoder's accuracy, paying the cross-encoder's cost on only 25 chunks instead of millions.

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

cross_encoder = HuggingFaceCrossEncoder(
    model_name="cross-encoder/ms-marco-MiniLM-L-6-v2"
)
reranker = CrossEncoderReranker(model=cross_encoder, top_n=4) # keep top 4

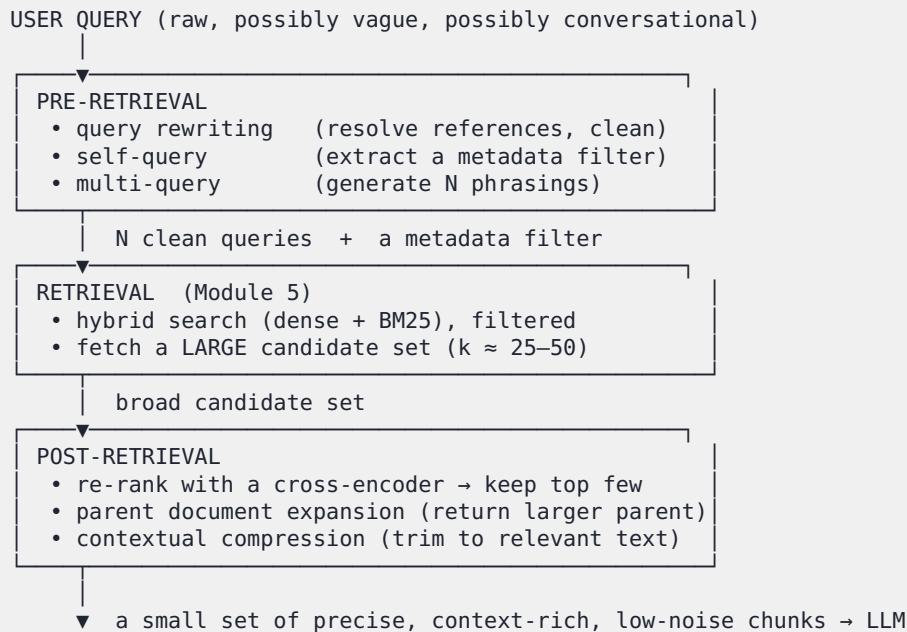
# Stage 1 retrieves broadly (k=25); Stage 2 re-ranks down to 4
rerank_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=vector_store.as_retriever(search_kwargs={"k": 25}),
)
chunks = rerank_retriever.invoke("How is annual leave accrued?")
```

### BEST PRACTICE

Two-stage retrieve-then-rerank is one of the highest-return upgrades in all of RAG, and it is a near-default for serious systems. The pattern: have stage 1 fetch *generously* (a large `k`, often 20–50) to maximise the chance the right chunk is *somewhere* in the candidate set, then let the cross-encoder do the precise sorting. Combine it with hybrid search from Module 5 — hybrid retrieval for a strong, diverse candidate set, then a cross-encoder re-ranker to put the truly best chunks on top — and you have the retrieval backbone of most production RAG systems, including the Module 10 capstone.

## 6.12 Combining Techniques and Enterprise Retrieval Systems

These techniques are layers, not alternatives, and a real enterprise retriever stacks several of them. A strong, representative enterprise retrieval pipeline looks like this:



Beyond stacking techniques, **enterprise retrieval systems** add concerns that go past the retrieval algorithm itself: **access control**, so the retriever returns only chunks a given user is permitted to see (enforced through metadata filters, building on Module 4's production architecture); **multi-tenancy**, keeping each customer's documents strictly isolated; **observability**, logging every transformed query and every retrieved chunk so retrieval failures can be diagnosed; **caching**, since query transformation and re-ranking add LLM calls whose results can often be reused; and **latency budgets**, because each technique here adds milliseconds and a production system must stay within a responsiveness target.

#### WHY THIS MATTERS

No single technique in this module is "the answer." The skill — and what interviewers and employers actually value — is *judgment*: knowing which layers a given problem needs and what each costs. Conversational system? Query rewriting is mandatory. Corpus rich in dates and categories? Self-query. Tight latency budget? Prefer the cheap `EmbeddingsFilter` over LLM-based compression. Quality is paramount and latency is negotiable? Stack multi-query, hybrid retrieval, and cross-encoder re-ranking. Advanced retrieval is a toolbox, and engineering is choosing the right tools for the job in front of you.

## 6.13 Workflow

The mental model for applying this module:

1. **Diagnose the failure.** Is retrieval missing relevant chunks (a *recall* problem) or returning noisy/irrelevant ones (a *precision* problem)? Is the issue the query or the documents?

2. **For query problems** (vague, conversational, vocabulary-mismatched, or constraint-laden queries) reach for **pre-retrieval** techniques: rewriting, multi-query, or self-query.
3. **For document problems** (chunks too small for context, too noisy, or mis-ordered) reach for **post-retrieval** techniques: parent document retriever, contextual compression, or re-ranking.
4. **Layer deliberately**, adding techniques one at a time and measuring the effect of each (Module 9) — never all at once, or you cannot tell what helped.
5. **Account for the cost.** Every technique here adds latency, and LLM-based ones add expense. Keep the system within its latency and cost budgets.

## 6.14 Step-by-Step Implementation and Code

---

The following file demonstrates the four headline techniques side by side. Save it as `module6_advanced_retrieval.py`.

```

"""
module6_advanced_retrieval.py
Advanced retrieval: multi-query, self-query, parent document, re-ranking.
Stack: Python + LangChain + Gemini.
Install: pip install langchain langchain-community langchain-google-genai \
        faiss-cpu lark sentence-transformers python-dotenv
"""

import os
from dotenv import load_dotenv
from langchain_google_genai import (
    GoogleGenerativeAIEmbeddings, ChatGoogleGenerativeAI)
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document

from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers import (
    ParentDocumentRetriever, ContextualCompressionRetriever)
from langchain.storage import InMemoryStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

# Knowledge base with rich metadata for the self-query demo
docs = [
    Document(page_content="Remote staff may claim a home-office stipend.",
            metadata={"department": "HR", "doc_type": "policy", "year": 2024}),
    Document(page_content="The 2022 budget review of engineering spend.",
            metadata={"department": "Finance", "doc_type": "report", "year": 2022}),
    Document(page_content="Annual leave for full-time staff is 24 days.",
            metadata={"department": "HR", "doc_type": "policy", "year": 2021}),
]
vector_store = FAISS.from_documents(docs, embeddings)
base_retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# ----- 1. MULTI-QUERY RETRIEVAL -----
multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever, llm=llm
)

# ----- 2. SELF-QUERY RETRIEVER (metadata-aware) -----
metadata_field_info = [
    AttributeInfo(name="department",
                  description="HR, Finance, or Engineering", type="string"),
    AttributeInfo(name="doc_type",
                  description="policy, report, or memo", type="string"),
    AttributeInfo(name="year",
                  description="publication year", type="integer"),
]
self_query_retriever = SelfQueryRetriever.from_llm(
    llm, vector_store, "Internal company documents", metadata_field_info
)

# ----- 3. PARENT DOCUMENT RETRIEVER -----
parent_store = FAISS.from_documents( # placeholder vector store for children
    [Document(page_content="seed")], embeddings)
parent_retriever = ParentDocumentRetriever(
    vectorstore=parent_store,
    docstore=InMemoryStore(),
    child_splitter=RecursiveCharacterTextSplitter(chunk_size=200),
    parent_splitter=RecursiveCharacterTextSplitter(chunk_size=800),
)

```

```

parent_retriever.add_documents(docs)

# ----- 4. RE-RANKING (two-stage retrieve-then-rerank) -----
reranker = CrossEncoderReranker(
    model=HuggingFaceCrossEncoder(
        model_name="cross-encoder/ms-marco-MiniLM-L-6-v2"),
    top_n=2,
)
rerank_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=vector_store.as_retriever(search_kwargs={"k": 3}),
)

def show(label, retriever, query):
    print(f"\n=== {label} === query: '{query}'")
    for i, doc in enumerate(retriever.invoke(query), 1):
        print(f"  {i}. {doc.page_content} | {doc.metadata}")

if __name__ == "__main__":
    show("Multi-query", multi_query_retriever, "how do I take time off")
    show("Self-query", self_query_retriever,
         "HR policies published after 2023")
    show("Parent document", parent_retriever, "home office stipend")
    show("Re-ranked", rerank_retriever, "annual leave entitlement")

```

## 6.15 Line-by-Line Code Explanation

**Setup.** The Gemini embedding model and chat model are created once; `temperature=0` makes the LLM's query transformations deterministic and focused. The three `Document` objects deliberately carry rich metadata (`department`, `doc_type`, `year`) so the self-query demo has real fields to filter on.

**Technique 1 — multi-query.** `MultiQueryRetriever.from_llm` wraps the base retriever and is given the LLM. When invoked with the vague query *"how do I take time off"*, it first asks the LLM to generate several rephrasings (e.g. about "annual leave" and "vacation policy"), runs the base retriever for each, and returns the pooled, de-duplicated results — so the leave-policy chunk is found even though the user's wording never said "leave."

**Technique 2 — self-query.** `metadata_field_info` declares the schema: three `AttributeInfo` entries telling the LLM exactly which fields exist, what they mean, and their types. `SelfQueryRetriever.from_llm` builds the retriever. When invoked with *"HR policies published after 2023"*, the LLM decomposes it into a semantic query and a structured filter — roughly `department = "HR" AND doc_type = "policy" AND year > 2023` — and runs a filtered similarity search. The 2024 HR policy passes the filter; the 2021 HR policy and the 2022 Finance report are filtered out before semantic search even runs. (This retriever requires the `lark` package, which parses the LLM-generated filter.)

**Technique 3 — parent document.** Two splitters are configured: a small `child_splitter` (200 chars, for precise search) and a larger `parent_splitter` (800 chars, for context). `ParentDocumentRetriever` is given a vector store (which will hold the embedded *children*)



and an `InMemoryStore` docstore (which holds the *parents*). `add_documents(docs)` splits each document both ways automatically. At query time, retrieval matches a small child chunk but returns its larger parent — "search small, return big."

**Technique 4 — re-ranking.** `HuggingFaceCrossEncoder` loads a cross-encoder model that scores query-chunk *pairs* jointly. `CrossEncoderReranker(..., top_n=2)` wraps it to keep the best 2. It is plugged in as the `base_compressor` of a `ContextualCompressionRetriever` whose `base_retriever` fetches a broader candidate set (`k=3` here; in production this would be 20–50). This is the two-stage pattern: stage 1 (the bi-encoder vector retriever) fetches broadly; stage 2 (the cross-encoder) re-ranks precisely.

**The `show` helper and tests.** `show` runs a retriever and prints each chunk with its metadata. The `__main__` block exercises each technique on a query chosen to highlight it: a vague query for multi-query, a constraint-laden query for self-query, a query needing context for parent document, and a query needing precise ordering for re-ranking.

#### COMMON MISTAKE

Stacking every advanced technique at once on the first attempt — multi-query plus self-query plus parent document plus compression plus re-ranking — and then being unable to tell which one helped, which one hurt, and where all the latency went. Add techniques **one at a time**, measure the effect of each (Module 9), and keep only those that earn their cost. A retriever with two well-chosen layers usually beats one with five thrown together blindly.

## 6.16 Common Mistakes

- **Trusting the raw user query.** Real queries are vague, conversational, and vocabulary-mismatched. Without query transformation, retrieval starts from a weak input.
- **Forgetting query rewriting in conversational RAG.** Follow-up questions full of pronouns ("what about the annual one?") are meaningless to a retriever unless rewritten with conversation context.
- **Expecting self-query to filter on absent metadata.** The LLM can only filter on fields you actually attached during ingestion and declared in the schema.
- **Choosing chunk size as a painful compromise.** The parent document retriever dissolves the precision-versus-context trade-off — use it instead of agonising over one chunk size.
- **Feeding whole noisy chunks to the model.** Without contextual compression, the answer arrives buried in irrelevant text, costing tokens and focus.
- **Skipping re-ranking, then blaming the embedding model.** Bi-encoder retrieval is coarse; a cross-encoder re-rank on a broad candidate set is often the real fix for "the right chunk was retrieved but ranked too low."

- **Stacking every technique blindly.** Each adds latency and (often) LLM-call cost. Add and measure one at a time.
- **Ignoring latency and cost budgets.** Multi-query, self-query, LLM compression, and re-ranking all add overhead — a production system must stay within its responsiveness target.

## 6.17 Best Practices

---

- **Diagnose before treating** — decide whether the failure is a query problem or a document problem, a recall problem or a precision problem.
- **Make query rewriting mandatory for conversational RAG**, folding in chat history.
- **Use multi-query** when users phrase the same need in many different ways.
- **Use the self-query retriever** for corpora with meaningful structured metadata — and design that metadata at ingestion time (Module 2).
- **Use the parent document retriever** to get precise retrieval and rich context at once.
- **Add contextual compression** to cut noise and token cost; prefer the cheap `EmbeddingsFilter` under tight latency budgets.
- **Adopt two-stage retrieve-then-rerank** as a near-default — fetch broadly, re-rank precisely.
- **Layer techniques deliberately**, one at a time, measuring each, and respecting latency and cost budgets.

## 6.18 Real-World Applications

---

- **Conversational assistants** — query rewriting makes multi-turn follow-up questions work; without it, chat-style RAG breaks on the second question.
- **Enterprise document search** — the self-query retriever lets employees ask "show me Finance reports from last quarter" in plain language and have it become a real filter.
- **Legal and research platforms** — multi-query widens recall across varied terminology; re-ranking ensures the most on-point ruling or paper surfaces first.
- **Technical documentation assistants** — the parent document retriever pinpoints the precise paragraph but returns the whole section so the answer makes sense.
- **Customer support** — contextual compression strips long help articles down to the few sentences that answer the ticket, lowering cost and sharpening answers.
- **High-stakes RAG (finance, medicine)** — two-stage retrieve-then-rerank maximises the chance the single most relevant, authoritative chunk is the one the model reasons over.

## 6.19 Mini Project: An Advanced Retrieval Upgrade

---

Take the retriever lab from Module 5 and upgrade it into an enterprise-grade retriever:

1. Start from a knowledge base whose chunks carry real metadata — at minimum a `source`, a `doc_type`, and a `year` or date field.
2. **Add query rewriting.** Build a rewrite chain and confirm that a deliberately messy, conversational query is cleaned up before retrieval. Test it with a follow-up question that only makes sense given a previous turn.
3. **Add multi-query retrieval.** Take a vague query that the Module 5 retriever handled poorly and show that multi-query recovers the right chunk. Print the generated query variations so you can see what the LLM produced.
4. **Add a self-query retriever.** Declare your metadata schema and verify that a question containing a date or category constraint produces a correct filter and excludes the documents that violate it.
5. **Add two-stage re-ranking.** Have stage 1 fetch a generous candidate set (k around 20) and a cross-encoder re-rank it down to the top 4. Find a query where re-ranking visibly improves the ordering versus plain retrieval.
6. **Compose them.** Build one retriever that does query rewriting, then hybrid retrieval, then re-ranking, end to end. Run it against your hardest test queries.
7. **Measure and decide.** For each technique, note its effect on result quality and its added latency. Write a short recommendation on which layers you would keep for *this* knowledge base, and why — the engineering-judgment skill this module is really teaching.

## 6.20 Chapter Summary

---

- Advanced retrieval removes two assumptions of the basic retriever: that the user's query is a good search query, and that retrieved chunks should be used as-is. Every technique acts either **pre-retrieval** (improve the query) or **post-retrieval** (curate the documents).
- **Multi-query retrieval** uses an LLM to generate several phrasings of the query, searches each, and pools the results — directly improving *recall*.
- **Query rewriting** cleans a single query (resolving references, removing filler, matching document vocabulary); **query expansion** adds related terms. Rewriting is essential for conversational RAG, where follow-up questions must be rewritten with chat history.
- The **self-query retriever** uses an LLM to decompose a natural-language question into a *semantic query* plus a *structured metadata filter*, then runs a filtered similarity search. It bridges semantic search and database querying, and requires a declared metadata schema — so metadata must be attached at ingestion.

- The **parent document retriever** dissolves the chunk-size dilemma: it embeds and searches *small child chunks* (precise retrieval) but returns their *larger parent chunks* (rich context). "Search small, return big." It uses two stores — a vector store for children, a document store for parents.
- **Contextual compression** trims or filters retrieved chunks against the query, keeping only relevant content. Compressors: `LLMChainExtractor` (extracts relevant sentences), `LLMChainFilter` (drops irrelevant chunks), `EmbeddingsFilter` (fast, no LLM).
- **Re-ranking** uses a **cross-encoder** — which scores query-chunk pairs jointly and accurately but slowly — in a **two-stage** pattern: a fast bi-encoder retrieves a broad candidate set, then the cross-encoder re-ranks it down to the best few. Retrieve-then-rerank is a near-default for production.
- These techniques **compose into layers**. Enterprise retrieval stacks several and adds access control, multi-tenancy, observability, caching, and latency budgets. The real skill is judgment: choosing the right layers for the problem, added and measured one at a time.

## 6.21 Practice Exercises

---

1. **Conceptual.** Explain the difference between pre-retrieval and post-retrieval techniques, and give one example of each, in under 120 words.
2. **Diagnosis.** For each symptom, name the most appropriate technique and say whether it is pre- or post-retrieval: (a) follow-up questions in a chat fail; (b) the right chunk is retrieved but ranked 8th; (c) users ask the same thing in many wordings and recall is poor; (d) retrieved chunks are too small to give the model enough context.
3. **Self-query.** Write three example user questions over a corpus of research papers with metadata `year`, `field`, and `citations`. For each, state the semantic query and the metadata filter a self-query retriever should produce.
4. **Parent document.** Explain, in your own words, why embedding small chunks but returning large parents gives both precise retrieval and rich context. Reference the "one chunk, one vector" principle from Module 3.
5. **Re-ranking.** Explain the difference between a bi-encoder and a cross-encoder, why each is suited to a different stage, and why you would not use a cross-encoder to search a million-chunk corpus directly.
6. **Code.** Run `module6_advanced_retrieval.py`. For the multi-query retriever, modify it to also print the LLM-generated query variations, and report what they were for a vague query of your choice.
7. **Trade-off analysis.** You must add advanced retrieval to a system with a strict 800-millisecond latency budget. Rank the techniques in this module from cheapest to most expensive in added latency, and recommend which you would and would not use.

## 6.22 Interview Questions

---

### BEGINNER

**Q1. What is the difference between pre-retrieval and post-retrieval techniques?**

Pre-retrieval techniques transform the user's query before the search runs (e.g. multi-query, query rewriting, self-query). Post-retrieval techniques transform or re-order the retrieved documents after the search runs (e.g. parent document retriever, contextual compression, re-ranking).

**Q2. What is multi-query retrieval?**

A technique that uses an LLM to generate several alternative phrasings of the user's question, runs retrieval for each, and pools the de-duplicated results. It improves recall by not betting everything on the user's single phrasing.

**Q3. What is query rewriting and when is it especially important?**

Using an LLM to turn a messy query into a clean one — resolving references, removing filler, matching document vocabulary. It is especially important for conversational RAG, where follow-up questions full of pronouns must be rewritten using the chat history to be searchable at all.

**Q4. What is contextual compression?**

A post-retrieval step that, given the query, trims or filters retrieved chunks so only genuinely relevant content reaches the model — reducing noise and token cost.

**INTERMEDIATE****Q5. What is the self-query retriever and what problem does it solve?**

It uses an LLM to decompose a natural-language question into a semantic query and a structured metadata filter, then runs a filtered similarity search. It solves the problem that constraints like dates, categories, and numeric thresholds are filtering conditions, not semantic ones — a plain retriever treats "after 2023" as vague meaning, while the self-query retriever turns it into a real filter. It needs a declared metadata schema.

**Q6. Explain the parent document retriever and the problem it solves.**

It solves the chunk-size dilemma: small chunks retrieve precisely but lack context, large chunks have context but retrieve imprecisely. The parent document retriever embeds and searches small child chunks (precision) but returns their larger parent chunks (context) — "search small, return big" — using a vector store for children and a separate document store for parents.

**Q7. What is the difference between a bi-encoder and a cross-encoder?**

A bi-encoder embeds the query and each chunk separately and compares them by a distance metric — fast and scalable, because chunk vectors are precomputed, but coarse. A cross-encoder takes the query and a chunk together as one input and outputs a precise relevance score — far more accurate but slow, because nothing can be precomputed and it must run per chunk.

**Q8. What are the three common types of contextual compressor?**

An extractive compressor (`LLMChainExtractor`) that uses an LLM to extract only the query-relevant sentences from each chunk; a filtering compressor (`LLMChainFilter`) that uses an LLM to keep or drop each chunk as a whole; and an embeddings filter (`EmbeddingsFilter`) that drops chunks below an embedding-similarity threshold without any LLM call — fast and cheap but coarser.

**ADVANCED****Q9. Describe two-stage retrieve-then-rerank and why it works.**

Stage 1 uses a fast bi-encoder retriever to fetch a large candidate set (e.g. top 25), prioritising recall cheaply. Stage 2 runs a slow, accurate cross-encoder on just those candidates, scoring each and keeping the best few, prioritising precision. It works because it pays the cross-encoder's high per-pair cost on only a small shortlist rather than the whole corpus, combining the bi-encoder's scalability with the cross-encoder's accuracy. The key tuning point is to make stage 1 fetch generously so the right chunk is somewhere in the candidate set.

**Q10. How would you design retrieval for a conversational, multi-tenant enterprise RAG system?**

Pre-retrieval: query rewriting that folds in conversation history so follow-ups are searchable, and optionally multi-query and a self-query filter. Retrieval: hybrid (dense + sparse) search restricted by metadata filters that enforce both relevance and access control — each tenant's and user's permitted documents only. Post-retrieval: fetch a broad candidate set, re-rank with a cross-encoder, optionally expand to parent documents, and compress to cut noise. Around all of it: strict tenant isolation, observability logging transformed queries and retrieved chunks, caching of repeated transformations, and a latency budget governing how many layers can run.

**Q11. Each advanced technique adds latency and often an LLM call. How do you decide which to use?**

By diagnosis and measurement. Identify whether the failure is a query problem or a document problem, and a recall problem or a precision problem, then apply the matching technique. Add techniques one at a time, measure the quality gain (Module 9) and the latency cost of each, and keep only those whose benefit justifies their cost within the system's latency and budget constraints. Under tight latency, prefer cheap options (the embeddings filter over LLM compression); when quality dominates, stack multi-query, hybrid retrieval, and cross-encoder re-ranking.

**SCENARIO-BASED**

**Q12. Your RAG chatbot answers the first question in a conversation well, but consistently fails on follow-up questions like "and what about the part-time staff?" Why, and how do you fix it?**

The follow-up query, on its own, is meaningless to the retriever — "what about the part-time staff?" has no topic; it depends entirely on the previous turn. The retriever embeds that contextless fragment and retrieves noise. The fix is a query-rewriting step that runs before retrieval and uses the conversation history to expand the follow-up into a self-contained query — for example, rewriting it to "what is the annual leave policy for part-time staff?" given that the prior turn was about annual leave. Query rewriting with chat history is effectively mandatory for any conversational RAG system.

**Q13. Users of your enterprise knowledge assistant complain that asking "show me the latest Finance reports" returns old documents and documents from other departments. The chunks all have `department` and `year` metadata. What is wrong and how do you fix it?**

The system is using plain semantic retrieval, so "latest" and "Finance" are being treated as fuzzy meaning rather than as real constraints — the retriever embeds the whole sentence and matches on overall similarity, with no actual filter on `department` or `year`. The fix is a self-query retriever: declare the `department` and `year` fields in a metadata schema, and the LLM will decompose the question into a semantic query ("reports") plus a structured filter (`department = "Finance"`, ordered or filtered by recent `year`), running a filtered similarity search that excludes other departments and old documents. This works only because the metadata was attached at ingestion time, which is why metadata schema design (Module 2) matters.

**END OF MODULE 6**

Your retriever is now genuinely intelligent — it improves the query before searching and curates the documents after. **Module 7 — Advanced RAG Patterns** assembles these retrievers into complete, named architectures: RAG Fusion and Reciprocal Rank Fusion, HyDE (Hypothetical Document Embeddings), Corrective RAG, Self-RAG, Graph RAG, and Multi-modal RAG. Where this module sharpened the *retriever*, Module 7 redesigns the *whole RAG pipeline* around smarter retrieval and self-correction.